PSI 2025-2026 – Objectif 1 : Implémentation d’un graphe en C#

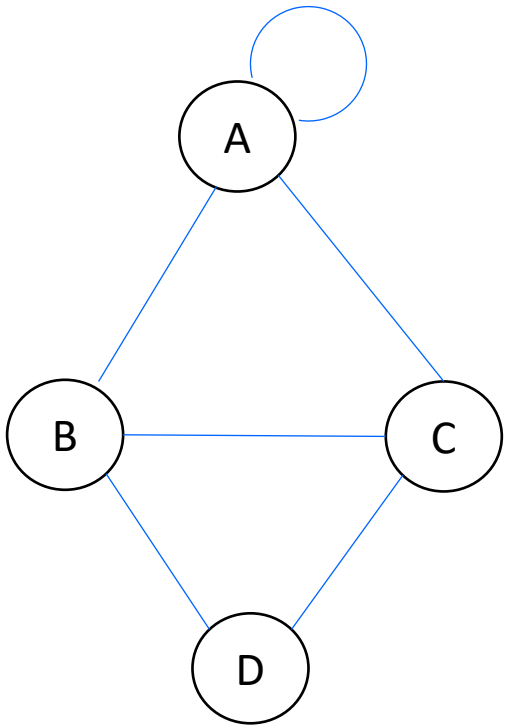
Michaël Soulignac

ESILV Nantes

Représentation d’un graphe basée sur une matrice d’adjacence

Matrice d'adjacence

Graphe non orienté



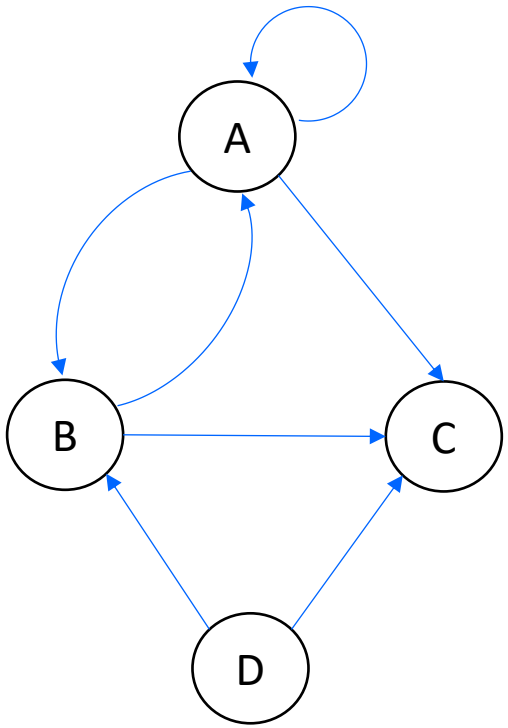
Arêtes

Matrice d'adjacence
(symétrique)

	A	B	C	D
A	1	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Matrice d'adjacence

Graphe orienté



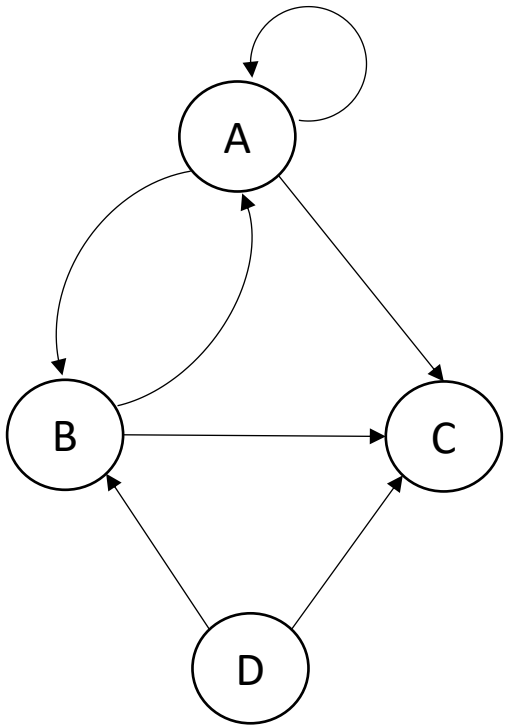
Arcs

Matrice d'adjacence

	A	B	C	D
A	1	1	1	0
B	1	0	1	0
C	0	0	0	0
D	0	1	1	0

Matrice d'adjacence

Graphe orienté



Matrice d'adjacence

Arcs

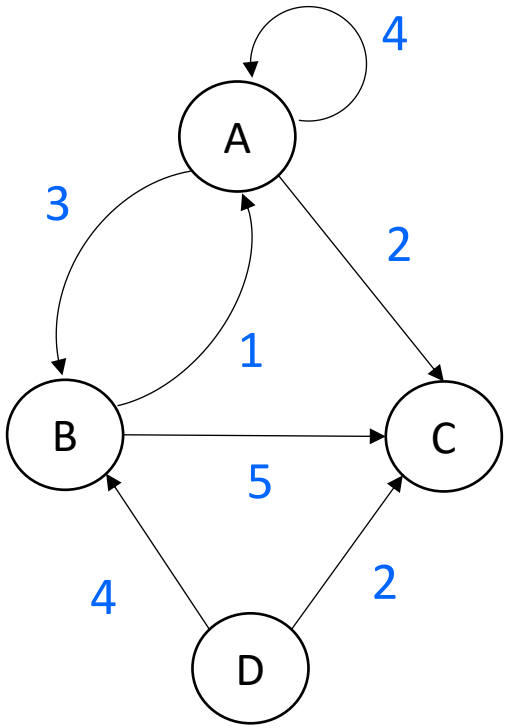
	A	B	C	D
A	1	1	1	0
B	1	0	1	0
C	0	0	0	0
D	0	1	1	0

Arcs sortants

Arcs entrants

Matrice d'adjacence

Graphe orienté et pondéré

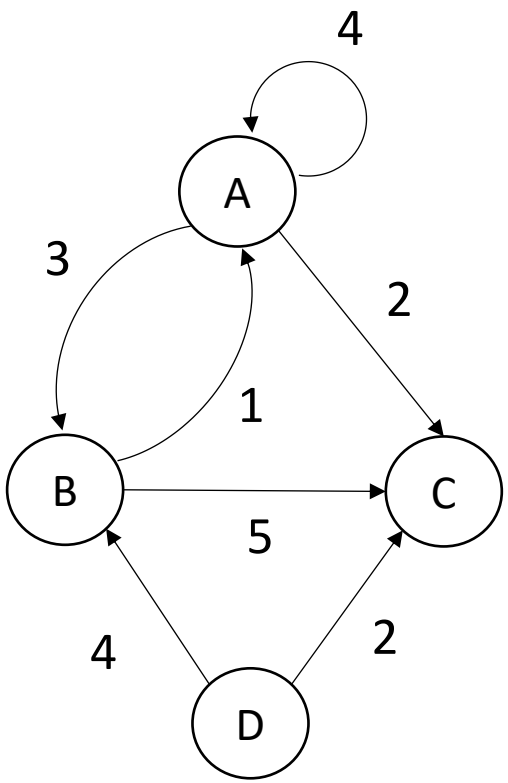


Arcs

Matrice d'adjacence

	A	B	C	D
A	**4**	**3**	**2**	0
B	**1**	0	**5**	0
C	0	0	0	0
D	0	**4**	**2**	0

Sommets et arcs



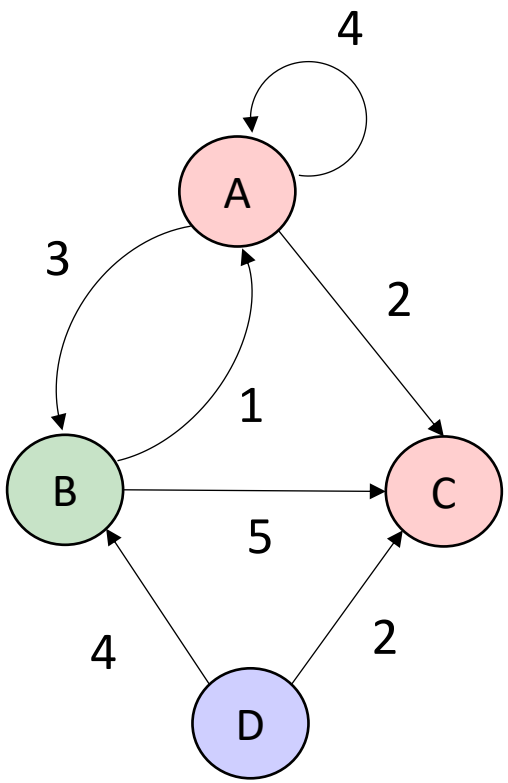
Sommets

0	1	2	3	← indice
A	B	C	D	← nom

Arcs

	0	1	2	3
0	4	3	2	0
1	1	0	5	0
2	0	0	0	0
3	0	4	2	0

Valeur d'un sommet



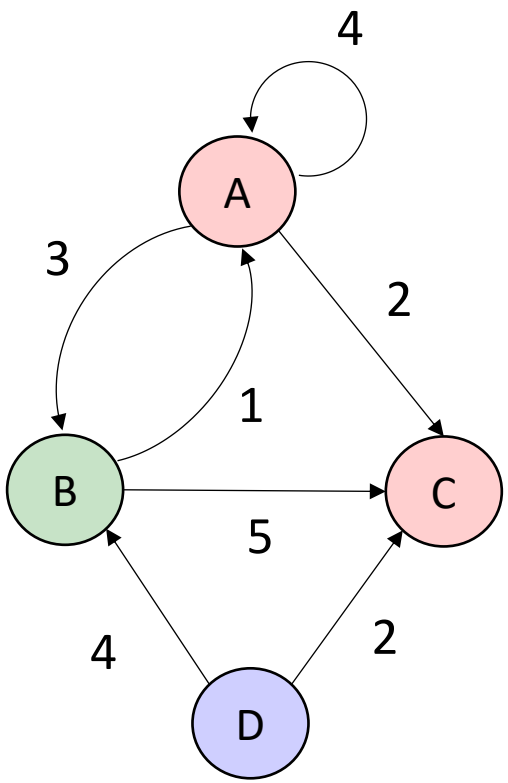
Sommets

0	1	2	3	← indice
A	B	C	D	← nom
2	1	2	3	← valeur

Arcs

	0	1	2	3
0	4	3	2	0
1	1	0	5	0
2	0	0	0	0
3	0	4	2	0

Concept de sommet



Sommets

0	1	2	3
A	B	C	D
2	1	2	3

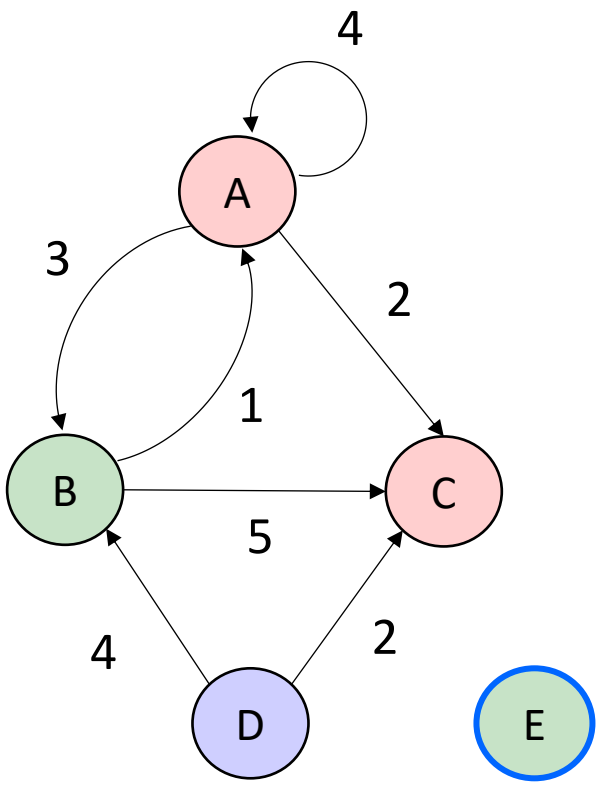
Sommet

nom
valeur

Arcs

	0	1	2	3
0	4	3	2	0
1	1	0	5	0
2	0	0	0	0
3	0	4	2	0

Ajout d'un sommet



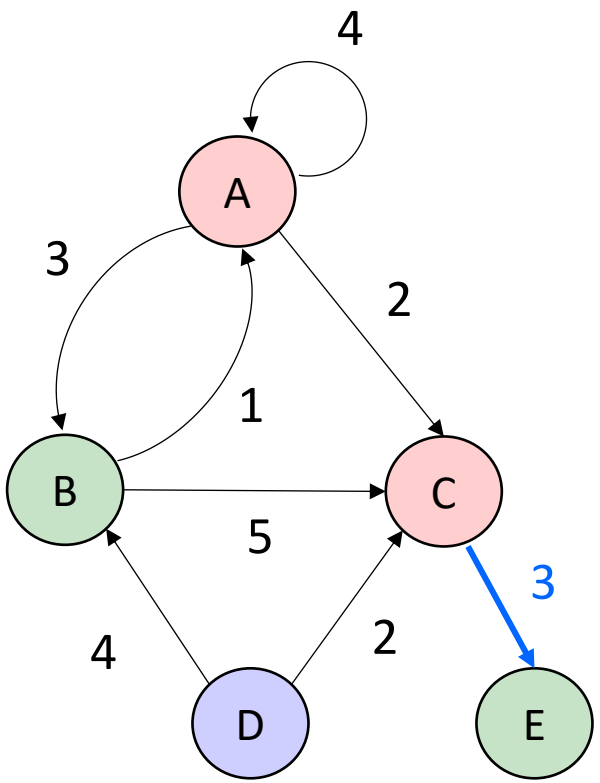
Sommets

0	1	2	3	4
A	B	C	D	E
2	1	2	3	1

Arcs

	0	1	2	3	4
0	4	3	2	0	0
1	1	0	5	0	0
2	0	0	0	0	0
3	0	4	2	0	0
4	0	0	0	0	0

Ajout d'un arc



Sommets

0	1	2	3	4
A	B	C	D	E
2	1	2	3	1

Arcs

	0	1	2	3	4
0	4	3	2	0	0
1	1	0	5	0	0
2	0	0	0	0	**3**
3	0	4	2	0	0
4	0	0	0	0	0

Implémentation en C#

Objectifs de l'implémentation en C#

1. Appliquer les principes de la Programmation Orientée Objet (POO) :
   - Modélisation (diagramme de classes)
   - Encapsulation
   - Composition
   - Exceptions

Objectifs de l'implémentation en C#

1. Appliquer les principes de la POO
2. Utiliser les collections génériques en C#
   - (Re)découvrir les structures de données disponibles

   ⇒ Comment les combiner pour stocker les constituants du graphe ?

   - Choisir les structures sur des critères de complexité temporelle et de simplicité d'utilisation

Objectifs de l'implémentation en C#

1. Appliquer les principes de la POO
2. Utiliser les collections génériques en C#
3. Valider le bon fonctionnement des classes avec des tests unitaires
   - Mise en œuvre avec MSTest dans Visual Studio

Exemple : stockage des sommets

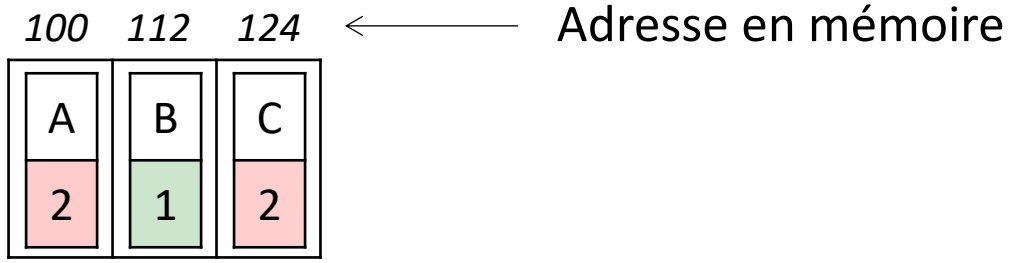
Quelques possibilités d'implémentation (avec des collections génériques) :

- List<Sommet>
- LinkedList<Sommet>
- SortedList<string, Sommet>
- Dictionary<string, Sommet>
- …

Exemple: tableau vs liste chaînée

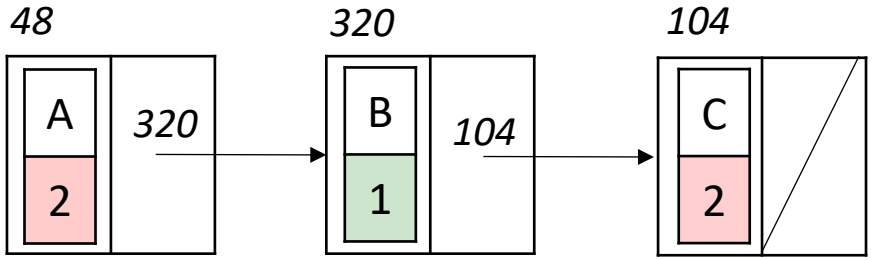
Tableau dynamique

`List<Sommet>`



Liste chaînée

`LinkedList<Sommet>`



⇒ Quelles conséquences sur les complexités temporelles pour :

- l'accès à un élément par son indice ?
- l'ajout / suppression d'un élément à un indice donné ?

Eléments fournis (*bootstrap*)

Eléments fournis pour démarrer

Une solution Visual Studio comprenant :

- Une implémentation partielle des classes `Graph` et `Matrix`, à compléter
- Une campagne de tests unitaires pour chaque classe

Modifications autorisées et interdites

- Interdit :
  - Renommer les classes ou les fichiers fournis
  - Modifier la signature des méthodes imposées (nom, types)
- Autorisé (et nécessaire !):
  - Implémenter le corps des méthodes imposées (commentaire `TODO : implémenter`)
  - Ajouter des attributs dans les classes fournies
  - Ajouter des méthodes dans les classes fournies
  - Ajouter d'autres classes, et les réutiliser dans les classes fournies

Exemple: extrait de la classe Graph

```
// Propriété : graphe orienté ou non
// Lecture seule
0 références
public bool Directed
{
    get;    // TODO : implémenter
    // pas de set
}


// --- Gestion des sommets ---

// Ajoute le sommet de nom `name` et de valeur `value` (0 par
// Lève une ArgumentException s'il existe déjà un sommet avec
0 références
public void AddVertex(string name, float value = 0)
{
    // TODO : implémenter
}
```

Parties modifiables

Exemple : un test unitaire de Graph

```
// --- Ajout d'un sommet ---

// Vérifie que AddVertex() ajoute un sommet avec la bonne valeur et incrémente l'ordre du graphe
// dans un graphe orienté ou non, selon la valeur de `directed`
✓ | 2 références | ✓ 1/1 passing
private void AddVertex_VertexAdded(bool directed)
{
    Graph g = new Graph(directed: directed);

    float aValue = 1;
    g.AddVertex("A", aValue);
    Assert.AreEqual(1, g.Order, "L'ordre du graphe est 1 après l'ajout de A");
    Assert.AreEqual(aValue, g.GetVertexValue("A"), $"La valeur de A est {aValue}");

    float bValue = 2;
    g.AddVertex("B", bValue);
    Assert.AreEqual(2, g.Order, "L'ordre du graphe est 2 après l'ajout de B");
    Assert.AreEqual(bValue, g.GetVertexValue("B"), $"La valeur de A est {bValue}");
}
```

Critères d’évaluation

Critères d'évaluation

- Implémentation fonctionnelle = note de l'objectif 1

⇒ Evalué après le rendu de l'objectif 1 par des tests unitaires

- Modélisation & étude = partie de la note de l'objectif 3 (rapport)
  - Choix et justification des structures de données
  - Diagramme de classes
  - Complexité des opérations du type `Graph`

⇒ Non évalué tout de suite, mais à anticiper (préparer les éléments)

Note de l'objectif 1

- Deux campagnes de tests unitaires: une pour `Matrix`, une pour `Graph`
- Note associée = 1 point par test réussi (20 tests par campagne)
- Poids : `Matrix` = 50 % de la note, `Graph` = 50%
- Si le code ne compile pas ⇒ Note = 0

⇒ Aucune surprise sur la note, nous la connaîtrez avant de rendre votre code